MULTI/DEX — A Natively Multi-Chain Decentralized Exchange

Built end-to-end on the Internet Computer. Real order book, real AMM, real on-chain oracle, real cross-chain ambitions.

In one paragraph

MULTI/DEX is a fully on-chain exchange that combines a central limit order book with a unified-basket automated market maker, fed by a multi-source on-chain oracle, defended by a protected-matching engine that disarms snipers, and accounted for through a single liquidity- provider token that represents a proportional claim on every asset the exchange holds. Frontend, backend, oracle, and matching engine all run inside Internet Computer canisters — no servers, no bridges to a trusted intermediary, no sequencer, no off-chain order matching. The "MULTI" in the name is a forward commitment: every architectural choice has been made so that the exchange can natively trade *any* asset the IC can hold a chain-key for — BTC, ETH, SOL, ICP today, and the long tail tomorrow.

Why this matters

DEXs broadly fall into two camps. **CLOB DEXs** give traders the experience they know from CEXs — limit orders, price-time priority, real liquidity providers — but typically by punting most of the engine off-chain to a sequencer. **AMM DEXs** are fully on-chain but stuck with the price-impact characteristics of a bonding curve, no actual order book, and an LP experience that's mostly impermanent loss.

The Internet Computer changes the constraint set. Compute is cheap enough and persistence is rich enough that you can run a **proper order book on-chain**, integrate an AMM into the same book as just another maker, and pull oracle data via HTTPS outcalls from real-world sources — all without giving up self-custody, censorship resistance, or auditability.

MULTI/DEX is what falls out when you take that constraint set seriously.

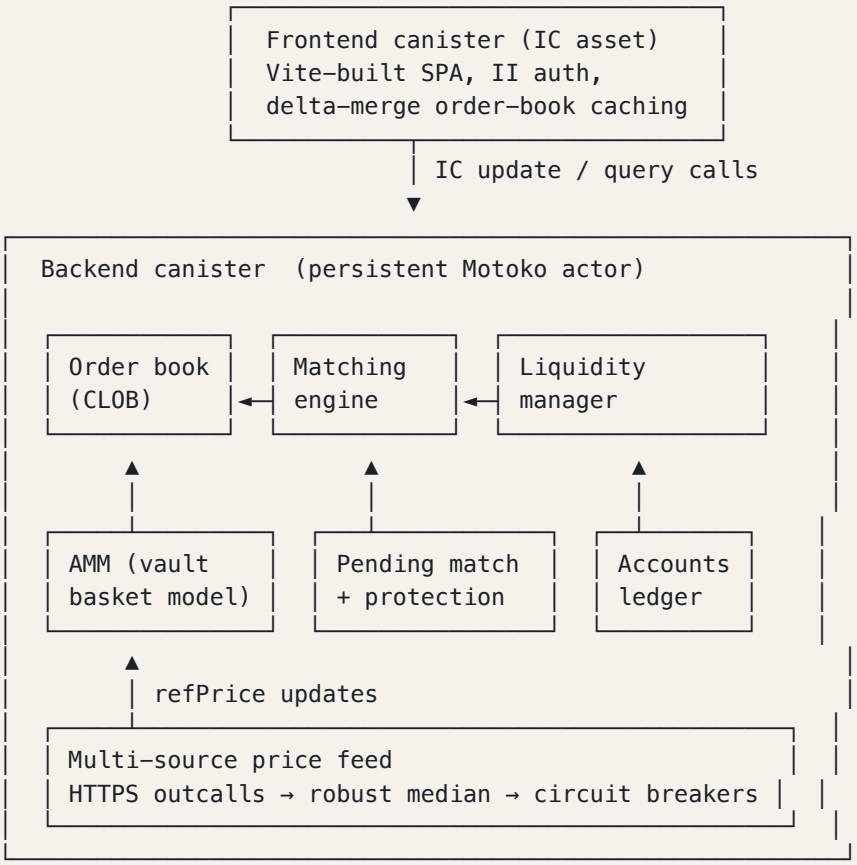
The five big ideas

1. **One vault, one LP token, many markets.** The AMM doesn't run as four siloed pools. It runs as a single multi-asset basket that targets a fixed **equal-weight** allocation — 12.5% each of BTC/ETH/SOL/ICP and 50% cash. LPs deposit any mixture of assets, mint one fungible LP token, and earn the spread captured across *every* market the AMM quotes in. P&L is reported at the basket level against the **total value ever deposited** (a cost-basis gain/loss, not a misleading mark-only line) — the way a real prime broker would account for it.
2. **A multi-source on-chain oracle with circuit breakers.** Price feeds come from several independent providers (Coinbase, CoinGecko, Coinpaprika, Kraken, CryptoCompare today, with Pyth Hermes and more coming) via HTTPS outcalls. Aggregation uses robust median with 2σ outlier trimming and needs at least two clean sources to price. On top of that: a **stale-feed pause** that freezes new quotes if the oracle goes dark, and a **sudden-jump circuit breaker** that refuses to apply a >2.5% move without a second confirming reading. Most DEX exploits over the last cycle were oracle exploits. This architecture closes those doors before they open.
3. **Sealed matching — the AMM is sniper-proof.** The AMM's quotes are *indicative*: they rest on the book for price discovery but are **non-takeable**. Every incoming order is sealed off-book and released only at the next *fresh* oracle price — so you can never fire at a number you've already seen and get filled at that now-stale price. By the time your order executes, the pool has re-fetched the oracle and re-quoted, and the AMM sweeps your resting order at *its* fresh price. This neutralises the latency-arbitrage attack that drains conventional on-chain AMMs every block — without a fast off-chain signing service. (It supersedes the earlier 8-second pending-match "void window"; the sealed release gives the same anti-snipe guarantee structurally, rather than relying on the AMM to react in time.)
4. **Two leverage modes: soft by default, opt-in cross-margin.** By default users place bids across multiple markets that exceed their cash in aggregate with **no debt** — the matching engine honours fills in order and shrinks/cancels the rest as cash is consumed, so there is nothing to liquidate. Opting into a **cross-margin account** then unlocks real leverage: your whole portfolio becomes LTV-weighted collateral, you borrow against it through a lazy linear-interest ledger, and a liquidation engine — partial-close to a health target, plus cross-market netting at the oracle mid — keeps the system solvent. A junior **insurance tranche** absorbs any insolvent bad debt before it can reach LPs. The soft model stays the default; debt is opt-in.
5. **It's all really running on-chain.** Frontend assets are served from an IC canister. The backend — order book, matching engine, AMM, oracle aggregator, vault accounting — is one persistent Motoko actor. State survives upgrades through Enhanced Orthogonal Persistence. There is no off-chain

database, no centralised sequencer, no Lambda function in the loop. A user can submit a trade directly to the IC and get a deterministic, replicated result.

How it actually works

The shape of the system



The whole system is roughly 10,000 lines of Motoko (a ~5,300-line core actor plus focused library modules for the order book, matching, AMM, oracle, margin, borrow, and liquidation). There are no microservices, no message queues, no cross-zone replication concerns — because the IC subnet *is* a replicated state machine. Everything is one synchronous state graph.

The four markets

Today MULTI/DEX runs four markets, all quoted in **ICPUSD** (a USD-pegged stable quote asset):

Market	Base	Quote	Base pooled (live, ~)
BTC-ICPUSD	BTC	ICPUSD	~4.5 BTC
ETH-ICPUSD	ETH	ICPUSD	~170 ETH
SOL-ICPUSD	SOL	ICPUSD	~4 300 SOL
ICP-ICPUSD	ICP	ICPUSD	~124k ICP

The base amounts drift with trading and rebalancing but are sized to the same **equal-weight** target — each asset ≈ 12.5% of the basket by USD value, the rest cash. Unified quote currency means cross-market trades and portfolio accounting are trivially expressible. The vault's total value is just `Σ basket[asset] × oracle[asset] + cash` — one number, one mark-to-market, one P&L curve.

The order book

A standard price-time-priority CLOB. Orders are stored as records keyed by id, indexed by market+side for fast snapshot construction, and maintained through every mutation by the matching engine. Snapshots aggregate orders into price levels on demand.

What makes the order book interesting in this codebase isn't the data structure — it's two things that hang off it:

- **Reservation-based balance management.** When a user places a buy, their ICPUSD is *reserved* (not transferred) until the order fills, is cancelled, or is shrunk. Reservations let us correctly bound a user's commitments across multiple in-flight orders without the double-spend race a naïve check-then-debit would produce.
- **Versioned change deltas.** Every mutation that affects a market's visible state — order created, order cancelled, order filled — bumps a per-market version counter and logs the affected price level. The frontend polls `getMarketChanges(lastVersion)` and gets back only the price levels that changed since its last poll. A busy market that would otherwise send 10 KB of snapshot every poll sends a few hundred bytes of delta instead. Most polls are sub-KB.

The AMM, in three layers

Layer 1 — Quote ladder generation. Given a `refPrice` from the oracle, an `inventoryTargetBase`, and the pool's current base holdings, the AMM constructs a ladder of bids and asks fanning out by `levelSpacingBps`. Each level has the same notional depth. The whole ladder shifts up or down by an **inventory skew** — long-base means the ladder drops (tighter asks, encouraging sales; worse bids, discouraging accumulation). Volatility regime (EWMA stdev of log- returns) further widens the spread when the market is hot.

Layer 2 — Indicative, non-takeable posting. Each ladder order rests on the book for price discovery but cannot be *taken* directly. An order that crosses the AMM is sealed off-book and released only once the pool has re-fetched the oracle and re-quoted; the AMM then sweeps the resting order at its own fresh price. This is the sniper defence: the latency arbitrageur who front-runs an oracle update never gets filled at the stale number he saw — by the time his order executes, the price has moved on. (Forced internal takers — the rebalancer and liquidation sales — likewise cannot take the AMM's own quotes, so their slippage is capped to the AMM's quoted band to stop them walking past the ladder into stranded book liquidity.)

Layer 3 — Active rebalancing. If the pool's inventory drifts more than 25% from target, the AMM takes the book — placing market orders on the side that brings inventory back. Rebalance size is bounded (5% of target per tick, capped), and there's a 60-second cooldown between actions. Rebalancing happens *automatically*, not on user demand; nothing about the trader UX depends on a rebalance succeeding.

The composite effect: a market maker that quotes confidently in calm markets, widens automatically in volatile ones, defends itself against toxic order flow, and pulls its inventory back to target without operator intervention.

The vault

This is the most novel piece of the design and it's worth dwelling on.

In a conventional AMM, each pool has its own pair of reserves and its own LP token. A BTC/USD pool's LP is one thing; an ETH/USD pool's LP is another. P&L is per-pool. If BTC outperforms ETH, BTC LPs win and ETH LPs lose — even though "the AMM" as a whole is doing fine.

MULTI/DEX's AMM has *one* set of reserves and *one* LP token. The basket holds BTC, ETH, SOL, ICP, and ICPUSD. When you deposit, the basket grows; you receive LP tokens in proportion to your contribution to the basket's pre-deposit value. When you withdraw, you burn LP and receive a proportional slice of *every* asset in the basket.

The basket targets a fixed **equal-weight** allocation — 12.5% each of BTC/ETH/SOL/ICP and 50% cash — hardcoded as the single source of truth (it also drives how much the AMM quotes per asset). What this gives us:

- **Two P&L numbers, both honest.** `valuePerLP = totalQuoteValue / lpSupply` is the per-share health line (anchored at 1.0, drifting up on captured spread and down on adverse selection). Alongside it, a **cost-basis gain/loss**: the vault tracks the *total value ever deposited* and reports `value / costBasis − 1`, so a freshly-seeded vault reads ≈ 0% rather than a misleading mark-only figure. No per-pool decomposition required.
- **Cross-market spread capture.** A trade on the BTC market and a trade on the SOL market both contribute spread to the same basket. LPs earn from the *system's* activity, not from any one pair.

- **Deposits priced to defend LPs.** LP minting is **fees-only**: a leg that would push the basket *further* from its target weight pays a quadratic fee, while a balancing (at-or-under-weight) leg mints at fair value — never a bonus. A mint-time bonus is extractable (deposit an under-weight asset cheap, withdraw the basket pro-rata, repeat), so there isn't one; the AMM's bid skew already pulls under-weight assets in through ordinary trading. A post-deposit **concentration cap**, a minimum first deposit, a virtual-share offset, and an oracle-freshness gate close the classic share-inflation and stale-price deposit attacks.
- **Natural multi-asset accounting.** Margin liquidations settle in basket form — opposing liquidatees are netted at the oracle mid, and cross-token seizures are absorbed straight into the basket rather than dumped on the book. The "receive a basket" infrastructure the vault was built around is exactly what margin settlement plugs into.

The oracle

Several independent sources, polled every 30 seconds via HTTPS outcalls, aggregated by **robust median with 2σ outlier rejection**. The aggregate is rejected if fewer than 2 sources returned a clean reading, or if cross-source disagreement exceeds 50 basis points. Successful readings update the AMM's `refPrice`; failures count toward the observability counters but leave the last-known-good `refPrice` in place. The AMM continues quoting on the old price.

On top of that, **Phase-6 hardening**:

- **Stale-feed pause.** If `refPrice` hasn't been refreshed for 5 minutes, the AMM stops accepting new quotes. After 10 minutes without an update, the AMM cancels its existing quotes entirely — off-market quotes are free arbitrage for whoever finds them first, and the only safe state when the oracle is dark is *off the book*.
- **Sudden-jump circuit breaker.** A single aggregate showing a

> *2.5% move from the previous `refPrice` is held in pending state. Only after a second aggregate confirms the new level (within 0.5%) is the move applied. This neutralises the single-source poisoning attack — a CDN-cached stale price or a briefly-compromised provider cannot push the AMM's mark in a single tick.*

Counters for both are exposed via `getPriceFeedStats`, so operators and users can see in real time how many refreshes succeeded, failed, were suspended, or triggered panic cancels.

Sealed matching, in detail

This is the mechanism that lets MULTI/DEX run a real AMM on-chain without bleeding to snipers.

Every order — market or limit — is **sealed off-book at submission**. Nothing matches synchronously. Instead:

1. The order is parked at its slippage cap (or limit price) and its funds are *reserved*, not moved. It appears in the trader's open orders immediately, but no trade has happened yet.
2. On the next oracle refresh the pool re-fetches its price and re-quotes ("GEPTOR" — get a fresh price, then requote). Only then is the parked order released — and only if the fresh price *postdates* the order (the anti-snipe gate). It then executes against the freshly-priced AMM plus any crossing user liquidity, up to its cap.
3. The AMM's own quotes are **non-takeable**: a released order fills user liquidity directly, and the AMM sweeps whatever rests against *its* fresh quote on the same cycle.

Why this is the right shape:

- **You cannot snipe a price you've already seen.** By the time your order executes, the AMM has re-priced. The latency arbitrageur's "free money" trade is gone before it can settle — structurally, not because a watcher reacted in time.
- **No fast off-chain signing service is needed.** The protection is fully on-chain and falls out of the ordinary oracle-refresh cycle (a ~1–2 s round trip), not a per-trade approval.
- **Graceful under a dark oracle.** If the feed stalls, parked orders fall back to executing against *user* liquidity only, clamped to a band around the last good mid, so the book keeps working without the AMM mispricing itself.

(The earlier design used an 8-second pending-match "void window" in which a protected maker could cancel a crossing after the fact. The sealed release supersedes it — same guarantee, without depending on the AMM to react inside a window, and applied to every order uniformly. A settlement-window API remains for client compatibility but no longer gates fills.)

Real-time data, delta-style

Conventional polling sends a full snapshot every poll. On an active order book this is wasteful — most polls find one or two levels changed, but the snapshot is the whole book.

MULTI/DEX's frontend keeps a local cache of each market's order book and asks the backend, on every poll, "what's changed since version X?" The backend returns just the affected price levels. The frontend merges them into the cache and renders the new top-of-book.

Typical poll payload: a few hundred bytes. Compared to a 10 KB snapshot, that's a >100× reduction. The UI stays smooth, the canister's egress stays cheap, and the experience is closer to a websocket feed than to traditional REST polling — without needing a websocket at all.

What you can do today

- **Sign in** with Internet Identity. A friendly auto-generated username represents your principal across the app.
- **Get test tokens** (in the demo deployment) and trade on any of the four markets.
- **Place limit, market, or protected limit orders.** Soft-leverage bidding works out of the box — express interest broader than your cash and let the matching engine handle consistency.
- **Become an LP.** Deposit any combination of BTC/ETH/SOL/ICP/ICPUSD into the vault, mint LP tokens (priced fees-only, so a balancing deposit mints at fair value), and watch your `valuePerLP` and cost-basis P&L. Withdraw at any time for a proportional slice of the live basket.
- **Trade on margin.** Open a cross-margin account, borrow against your whole portfolio, and go long or short with real leverage — with a live health bar, partial-close liquidations, and an insurance backstop.
- **Stake the insurance fund.** Provide an ICPUSD junior tranche that earns liquidation penalties and absorbs bad debt before LPs do.
- **Watch the AMM defend itself** on the Stats page: oracle feed health, per-pool inventory skew, volatility regime, P&L curve, and the share of the book the AMM is responsible for at any moment.

What's on the roadmap

These aren't promises, they're the next big architectural decisions we're actively working through:

- **Chain-key asset integration.** ckBTC, ckETH, ckUSDC, eventually chain-key SOL once the IC integration ships. Today's "BTC" in MULTI/DEX is a placeholder ledger entry; the next step is wiring it to the real chain-key Bitcoin canister so deposits and withdrawals bridge native BTC.
- **Margin trading.** *(Shipped.)* Opt-in portfolio cross-margin accounts (whole balance is LTV-weighted collateral; borrowed funds + the assets bought with them count, enabling real leverage), a lazy linear-interest borrow ledger, and a liquidation engine with partial-position close (de-risk to a 1.25 health target rather than full close) and cross-market netting that matches opposing liquidatees at the oracle mid before touching the book — backstopped by a junior insurance tranche. See src/backend/lib/{MarginEngine,BorrowEngine, Liquidator}.mo. **Next:** a *positions UX* that presents the borrow-and-spot mechanics as plain long/short positions — entry price, PnL/ROE, liquidation price, and a single "% to liquidation" figure (see `docs/margin-simplification-design.md`); plus TWAP residual slicing for large book-bound liquidations and basket-form payouts to margin providers.
- **More oracle sources.** XRC integration is gated on its update cadence; Pyth Hermes, Chainlink CCIP feeds, and additional redundancy via more HTTP providers are all viable.

- **Perpetual swaps.** Cash-settled, funding-rate-driven perps on the same four assets, sharing the vault and margin infrastructure.
- **Cross-protocol composability.** Other IC canisters calling MULTI/DEX's public API to route their own trades — programmatic liquidity discovery, native to the platform.

A note on why on-chain matters

It's easy to read this document and think the technical achievements are interesting but the on-chainness is a footnote. It's not.

When the order book runs on a single server, the operator can front- run users, censor orders, lose state to an outage, or get compelled by a regulator to freeze withdrawals. Users have to trust the operator on every one of those axes, and history shows that trust is sometimes misplaced — sometimes catastrophically.

When the order book runs on a replicated, deterministic, consensus- secured state machine — and the matching engine, the AMM, the oracle aggregator, and the user balances all run in the same process — you get something qualitatively different. The exchange's behaviour is **verifiable**. Its code is **auditable on-chain**. Its state cannot be selectively withheld from one user. Its upgrade path goes through the canister's controller, which can be governed by an NNS DAO, a multisig, or whatever decentralisation pattern the community chooses.

Every architectural choice in MULTI/DEX — the protected matching, the vault basket, the multi-source oracle, the soft-leverage matching semantics — was selected with this in mind. The DEX has to be good *because* it's on-chain, not despite it.

Tech stack, succinctly

- **Internet Computer** (subnet replicated, BLS-signed, certified state, HTTPS outcalls)
- **Motoko** with Enhanced Orthogonal Persistence (`persistent actor`, `mo:core 2.5.0`)
- **mops** for dependency management
- **Vite + vanilla JS** frontend, served from an IC asset canister
- **@icp-sdk/auth + @icp-sdk/core** for Internet Identity
- **Internet Identity** for authentication, with delegations capped at 8 hours
- **TradingView Lightweight Charts** for the markets page
- **CoinGecko, Coinbase, Coinpaprika** as live oracle sources

- **pocket-ic** for local development, mainnet-equivalent semantics

---

The takeaway

MULTI/DEX is a thesis bet that **a real exchange — order book, AMM, oracle, vault, accounting, frontend — can run end-to-end inside a modern smart contract platform without compromising on the trading experience**. Every piece is implemented to a standard that's comparable with serious centralised systems, while inheriting the properties that only on-chain deployment can provide.

The "MULTI" half of the name is the next chapter. With chain-key crypto on the IC, this exchange's natural endpoint isn't "another DEX on one chain" — it's a single, unified venue where assets from every major chain trade against each other natively. That's the direction every architectural choice in this codebase is pointed.

Today: four markets, several oracle sources, three trading modes (soft- leverage spot, opt-in cross-margin, and the AMM), one vault, zero off-chain components.

Tomorrow: many more of each.